

UNIVERSIDAD TÉCNICA NACIONAL

Ingeniería en Tecnologías de Información

PROYECTO FINAL

Sistema de Gestión de Licitaciones

Metodología Ágil: *Extreme Programming (XP)*

Curso	Metodologías Ágiles de Desarrollo de Software
Código del curso	ITI-822
Docente	Andrés Joseph Jiménez Leandro
Modalidad	Individual o pareja; máximo dos integrantes.
Periodo lectivo	IIC-2026
Fecha	Lunes 13 de Julio del 2026

Control del Documento

Elemento	Definición
Nombre del proyecto	Sistema de Gestión de Licitaciones
Naturaleza	Proyecto final de desarrollo de software y aplicación exclusiva de Extreme Programming (XP).
Modalidad de trabajo	Individual o en pareja. No se aceptan tríos ni grupos de más de dos personas.
Producto principal	Aplicación web modular en .NET 9 con ASP.NET Core MVC, API REST, PostgreSQL, Docker y Kubernetes.
Repositorio	GitHub, con historial incremental, integración continua y documentación interna en Markdown.
Valor de la evaluación	100 puntos.

Condición metodológica obligatoria

El proyecto debe desarrollarse exclusivamente mediante Extreme Programming (XP). No se permite combinar XP con Scrum, Kanban ni con marcos híbridos. No deben utilizarse roles, ceremonias o artefactos propios de Scrum o Kanban como metodología rectora del trabajo.

1. Presentación y propósito

El proyecto consiste en diseñar, construir, probar, contenerizar y desplegar una aplicación web para administrar licitaciones, proveedores, ofertas económicas, niveles de aprobación y conversión referencial de moneda. La solución deberá demostrar dominio técnico y aplicación disciplinada de Extreme Programming (XP) como única metodología ágil del proyecto.

El sistema utilizará el colón costarricense (CRC) como moneda oficial y fuente de verdad. La interfaz permitirá alternar la visualización de montos a dólares estadounidenses (USD) mediante un tipo de cambio administrable, sin modificar los valores originales almacenados.

El proyecto debe desarrollarse de forma incremental, con entregas pequeñas y frecuentes, pruebas automatizadas, integración continua, diseño simple, refactorización constante y evidencia verificable del proceso XP dentro del repositorio.

2. Objetivos

2.1. Objetivo general

Construir una solución web modular, mantenible, verificable y desplegable para gestionar licitaciones y ofertas, aplicando exclusivamente Extreme Programming (XP), buenas prácticas de ingeniería de software y una persistencia relacional basada en PostgreSQL.

2.2. Objetivos específicos

- Administrar licitaciones, proveedores, ofertas, niveles de aprobación y tipos de cambio mediante operaciones CRUD completas.
- Aplicar reglas de negocio e integridad de datos en interfaz, servidor y PostgreSQL.
- Exponer las operaciones principales mediante ASP.NET Core MVC y una API REST documentada.
- Aplicar Planning Game, historias de usuario, iteraciones cortas, TDD, integración continua, diseño simple, refactorización y propiedad colectiva del código.
- Ejecutar la solución con Docker Compose y desplegarla en Kubernetes con persistencia y configuración segura.
- Mantener toda la documentación técnica y metodológica dentro del repositorio, en la carpeta /docs y en formato Markdown.
- Demostrar el funcionamiento mediante pruebas unitarias, de integración y funcionales de extremo a extremo.

3. Modalidad y organización del trabajo

Aspecto	Requisito
Cantidad de integrantes	Una persona o una pareja. No se admiten tríos ni agrupaciones de más de dos integrantes.
Responsabilidad	Cada integrante debe comprender y poder defender cualquier módulo del proyecto.
Repositorio	Un único repositorio de GitHub por proyecto.
Trabajo en pareja	Debe existir programación en parejas con rotación frecuente de los roles.

Aspecto	Requisito
Trabajo individual	Se aplican todas las prácticas XP compatibles con la modalidad individual; las revisiones, TDD, refactorización e integración continua deben quedar evidenciadas.
Evaluación individual	La persona docente va a realizar una defensa oral, revisión del historial y modificación práctica en vivo para comprobar autoría y dominio.

Restricción de agrupación

No se aceptará una entrega desarrollada por tres o más personas, aunque el repositorio o la documentación distribuyan las tareas entre ellas.

4. Metodología Ágil a utilizar: *Extreme Programming (XP)*

El proceso de desarrollo será regido únicamente por Extreme Programming (XP). Los estudiantes deben utilizar terminología, prácticas y evidencias propias de XP. No se aceptará presentar el proyecto como Scrum, Kanban, ni como combinación de varios marcos ágiles.

4.1. Prácticas XP obligatorias

Práctica XP	Evidencia requerida
Planning Game/Meet	Definir historias de usuario, prioridad, estimación, criterios de aceptación, plan de liberación y plan de cada iteración.
Historias de usuario	Redactar las necesidades del proyecto, desde la perspectiva del cliente, con criterios verificables y vínculo a commits o pruebas (<i>en la documentación md dentro del proyecto</i>).
Iteraciones cortas	Realizar al menos tres iteraciones de duración uniforme.
Pequeñas liberaciones	Producir una versión ejecutable y demostrable al cierre de cada iteración.
TDD	Escribir primero una prueba que falle, implementar el mínimo código para aprobarla y luego refactorizar.
Programación en parejas	Obligatoria para proyectos de dos integrantes, con alternancia documentada de los roles.
Integración continua	Integrar cambios frecuentemente y mantener compilación, pruebas y análisis automáticos en estado satisfactorio. CI/CD con Kubernetes, desde GitHub usando GitHub Actions.
Diseño simple	Implementar la solución más sencilla que satisfaga las historias vigentes, evitando complejidad especulativa.
Refactorización	Mejorar continuamente estructura, legibilidad y diseño sin alterar el comportamiento observable.
Propiedad colectiva	En parejas, ambos integrantes pueden mejorar cualquier parte del código y son responsables del producto completo.
Estándares de código	Aplicar convenciones uniformes, nombres descriptivos, comentarios útiles y análisis estático.
Ritmo sostenible	Distribuir el trabajo durante las iteraciones; el historial no debe evidenciar que todo el proyecto se construyó al final.
Pruebas de aceptación	Cada historia terminada debe poder verificarse mediante criterios de aceptación y pruebas reproducibles.

4.2. Artefactos permitidos y prohibidos

Se utilizará en XP	No se utilizará como metodología
Historias de usuario, tarjetas o archivos Markdown.	Product Backlog, Sprint Backlog o Scrum Board.
Planning Game, plan de liberación y plan de iteración.	Sprint Planning, Daily Scrum, Sprint Review o Sprint Retrospective.
Cliente, programadores y pruebas de aceptación.	Product Owner, Scrum Master o equipo Scrum como roles del proceso.
Velocidad XP y entregas pequeñas.	Límites WIP, flujo Kanban o políticas de tablero Kanban como sistema rector.
Integración continua y refactorización.	Combinaciones híbridas como Scrumban.

GitHub Issues y Milestones pueden utilizarse para registrar historias, defectos e iteraciones, siempre que se empleen con el proceso XP descrito y no como sustitución por Scrum o Kanban.

4.3. Evidencias mínimas por iteración

- Historias seleccionadas para la iteración, prioridad, estimación y criterios de aceptación.
- Pruebas escritas o actualizadas antes o junto con la implementación, con evidencia del ciclo rojo-verde-refactorización.
- Commits frecuentes asociados con historias, pruebas, correcciones o refactorizaciones.
- Ejecución satisfactoria de la integración continua.
- Versión funcional demostrable.
- Registro breve de retroalimentación del cliente y ajustes para la siguiente iteración.
- Velocidad observada y comparación con la planificación, sin emplear terminología de sprint.

5. Alcance funcional

5.1. Página inicial y navegación

- Landing page que explique el propósito de la aplicación, el flujo de licitación, las ofertas, la mejor oferta, el nivel de aprobación y la conversión monetaria.
- Menú visible con acceso a Inicio, Licitaciones, Proveedores, Ofertas, Niveles de aprobación, Tipo de cambio y documentación interactiva de la API.
- Diseño adaptable para computadoras y dispositivos móviles.

5.2. CRUD completo

Módulo	Operaciones mínimas
Licitaciones	Crear, listar, consultar, editar, eliminar o aplicar borrado lógico, cambiar estado, consultar ofertas, mejor oferta, clasificación y aprobador.
Proveedores	Crear, listar, consultar, editar, eliminar o aplicar borrado lógico y consultar ofertas relacionadas.
Ofertas	Crear, listar, consultar, editar, eliminar y filtrar por licitación y proveedor, respetando estado y vencimiento.

Módulo	Operaciones mínimas
Niveles de aprobación	Crear, listar, consultar, editar y eliminar rangos no traslapados.
Tipos de cambio	Crear, listar, consultar, editar, eliminar y seleccionar el registro activo.

5.3. Flujo funcional mínimo

1. Abrir la landing page y consultar la explicación general.
2. Cambiar entre modo claro y modo oscuro.
3. Registrar un proveedor con nombre válido y único.
4. Crear una licitación con código único y seleccionar la fecha y hora de cierre mediante calendario.
5. Publicar la licitación mediante una transición permitida.
6. Registrar una oferta válida para el proveedor.
7. Intentar registrar una oferta duplicada, una oferta superior al presupuesto y una oferta vencida, verificando su rechazo.
8. Consultar la mejor oferta, su clasificación y el nivel de aprobación.
9. Alternar visualmente los montos entre CRC y USD.
10. Realizar las operaciones equivalentes mediante la API REST.
11. Ejecutar pruebas, Docker Compose, integración continua y despliegue en Kubernetes.

6. Tecnologías y arquitectura

6.1. Tecnologías obligatorias

- .NET 9.
- ASP.NET Core MVC.
- ASP.NET Core Web API.
- Entity Framework Core 9.
- PostgreSQL 16 o superior.
- HTML5, CSS3 y JavaScript.
- Bootstrap o biblioteca visual equivalente.
- Docker y Docker Compose.
- Kubernetes.
- Git, GitHub y GitHub Actions.
- xUnit, NUnit o MSTest.
- Playwright o Selenium para pruebas funcionales de navegador.
- Testcontainers o mecanismo equivalente para pruebas de integración con PostgreSQL real.

6.2. Estructura modular esperada

```

/src
  /Licitaciones.Domain
  /Licitaciones.Application
  /Licitaciones.Infrastructure
  /Licitaciones.Web
  /Licitaciones.Api
/tests
  /Licitaciones.UnitTests

```

```
/Licitaciones.IntegrationTests
/Licitaciones.FunctionalTests
/docs
/k8s
```

Proyecto o Carpeta	Responsabilidad
Domain	Entidades, objetos de valor, enumeraciones, contratos y reglas centrales sin dependencia de infraestructura.
Application	Casos de uso, servicios de aplicación, DTO, validadores y puertos o interfaces.
Infrastructure	Entity Framework Core, PostgreSQL, repositorios, migraciones, logging y servicios externos.
Web	Controladores MVC, vistas, navegación, temas, validación visual y experiencia de usuario.
API	Endpoints REST, contratos HTTP, OpenAPI, versionado y ProblemDetails.
Tests	Pruebas unitarias, de integración y funcionales separadas por responsabilidad.
docs	Única ubicación autorizada para la documentación del proyecto, en Markdown.
k8s	Manifiestos de Kubernetes para aplicación, base de datos, almacenamiento y configuración.

6.3. Monolito modular y microservicios

La solución debe ser modular. Puede implementarse como monolito modular o mediante microservicios cuando la separación esté técnicamente justificada. La elección de microservicios no modifica la ponderación de la evaluación ni sustituye la obligación de cumplir todos los requisitos funcionales, de pruebas, documentación y despliegue.

Cuando se utilicen microservicios, cada servicio debe poseer límites claros, comunicación documentada, configuración independiente y responsabilidades cohesionadas. No se aceptará dividir artificialmente el sistema para aparentar mayor complejidad.

6.4. Calidad del código

- Controladores delgados; la lógica de negocio debe residir en servicios o capas apropiadas.
- Uso de inyección de dependencias y abstracciones justificadas.
- Código comentado de forma útil y profesional, especialmente en reglas no evidentes.
- Documentación XML en clases y métodos públicos relevantes.
- Prohibición de comentarios redundantes, código muerto, archivos temporales y mensajes de depuración.
- Dependencias justificadas, actualizadas y sin paquetes innecesarios.
- Compilación sin advertencias evitables y formato uniforme.

7. Modelo de Datos

Entidad	Campos mínimos
Licitación	Id; Código; CódigoNormalizado; Título; Estado; FechaCierre; PresupuestoEstimadoCRC; CreatedAt; UpdatedAt; versión de concurrencia.
Proveedor	Id; Nombre; NombreNormalizado; CreatedAt; UpdatedAt; versión de concurrencia.

Entidad	Campos mínimos
Oferta	Id; LicitacionId; ProveedorId; MontoOfertadoCRC; FechaRegistro; UpdatedAt; versión de concurrencia.
Nivel de Aprobación	Id; MontoMinimoCRC; MontoMaximoCRC nullable; Aprobador; CreatedAt; UpdatedAt.
Tipo de Cambio	Id; CRCporUSD; FechaVigencia; Activo; CreatedAt; UpdatedAt.

Los identificadores deben ser generados automáticamente y no deben ser editables por el usuario. Los montos utilizarán decimal con precisión explícita, por ejemplo: *numeric(18,2)*; queda prohibido utilizar float o double para valores monetarios.

8. Reglas de negocio y validaciones

8.1. Ciclo de Estados

Estado Actual	Transición Permitida	Condición
Borrador 	<i>Publicada</i>	Datos completos, presupuesto válido y fecha de cierre futura.
Borrador 	<i>Cerrada</i>	Permitida como cancelación documentada.
Publicada 	<i>Cerrada</i>	Por acción autorizada o al alcanzar la fecha de cierre.
Publicada 	<i>Borrador</i>	No permitida.
Cerrada 	<i>Publicada o Borrador</i>	No permitida, salvo una regla de reapertura aprobada previamente por la persona docente.

Una licitación cuya fecha de cierre haya sido alcanzada se considera cerrada funcionalmente, aunque una actualización tardía del campo de estado todavía indique Publicada.

8.2. Fecha y Vencimiento

- La fecha y hora de cierre se seleccionarán mediante un control de calendario y hora, no únicamente mediante texto manual.
- No se aceptará una oferta cuando la fecha y hora actual sean iguales o posteriores a la fecha de cierre.
- Las fechas deben almacenarse mediante DateTimeOffset o una estrategia equivalente; las comparaciones internas se realizarán en UTC y la presentación utilizará America/Costa_Rica.
- Las ofertas vencidas o asociadas con licitaciones cerradas no pueden crearse, editarse ni eliminarse.
- El reloj debe abstraerse mediante un servicio inyectable para permitir pruebas deterministas.

8.3. Unicidad y Normalización

- El código de licitación debe ser único, ignorando espacios laterales y diferencias entre mayúsculas y minúsculas.
- El nombre del proveedor debe ser único después de eliminar espacios laterales, reducir espacios repetidos, normalizar Unicode y comparar sin distinguir mayúsculas y minúsculas.
- La unicidad debe validarse en interfaz, servidor y PostgreSQL mediante índices únicos.

- Un proveedor no puede registrar más de una oferta para la misma licitación; debe existir un índice único compuesto LicitacionId + ProveedorId.

Ejemplos equivalentes:

Empresa Central
empresa central
EMPRESA CENTRAL

8.4. Caracteres permitidos

El nombre del proveedor puede contener letras, números, espacios, punto, coma y paréntesis normales. No se permiten otros símbolos.

Expresión regular de referencia para .NET: `^[p{L}\p{N}.,\(\)]+$`

8.5. Valores numéricos y monetarios

- Presupuesto, oferta y tipo de cambio deben ser mayores que cero.
- No se aceptan números negativos ni el valor cero.
- Los campos numéricos deben utilizar controles apropiados y validación en cliente, servidor y base de datos.
- La oferta no puede superar el presupuesto. Una oferta igual al presupuesto es válida.
- No puede reducirse el presupuesto por debajo de una oferta existente.

8.6. Mejor oferta y clasificación

- La mejor oferta es la oferta válida con el menor monto en CRC.
- En empate, se selecciona la oferta registrada primero.
- Sin ofertas: “Sin ofertas válidas”.
- Ahorro igual o superior al 10 %: “Oferta conveniente”.
- Ahorro mayor que 0 % y menor que 10 %: “Oferta aceptable”.
- Oferta igual al presupuesto: “Oferta válida sin ahorro”.

Porcentaje de ahorro = $((\text{Presupuesto CRC} - \text{Mejor oferta CRC}) / \text{Presupuesto CRC}) \times 100$

8.7. Niveles de Aprobación

El aprobador debe obtenerse desde una tabla parametrizable y no mediante una cadena fija de condiciones if/else. Los rangos no pueden traslaparse y solo puede existir un rango abierto sin monto máximo.

Monto mínimo CRC	Monto máximo CRC	Aprobador
0,01	999 999,99	Encargado de área
1 000 000,00	9 999 999,99	Gerencia
10 000 000,00	Sin límite	Junta Directiva

8.8. Conversión CRC/USD

- Los valores oficiales se almacenan únicamente en CRC.
- La conversión a USD es una representación calculada y no modifica los valores persistidos.

- Debe mostrarse la fecha del tipo de cambio utilizado.
- Solo puede existir un tipo de cambio activo para la operación ordinaria.
- La solución debe funcionar sin Internet mediante un tipo de cambio administrable localmente.

Monto USD = Monto CRC / Tipo de cambio CRC por USD

8.9. Eliminación e Integridad

- No debe eliminarse físicamente una licitación o proveedor con ofertas relacionadas, salvo que se implemente borrado lógico de forma consistente.
- Los errores de integridad referencial deben traducirse a mensajes controlados.
- Las ofertas cerradas deben conservarse como evidencia y no deben alterarse.
- Debe solicitarse confirmación antes de cualquier eliminación permitida.

9. Interfaz, Accesibilidad y Experiencia de Usuario

- Modo claro y modo oscuro con control visible y persistencia de preferencia.
- Botón para alternar los montos entre CRC y USD.
- Mensajes de éxito, advertencia y error visibles y comprensibles.
- Formularios con validación junto al campo correspondiente.
- Tablas con paginación, filtrado y ordenamiento adecuados.
- Recursos front-end incluidos localmente o con un mecanismo documentado de respaldo; la interfaz no debe quedar inutilizable por falta de acceso a una CDN.
- Formato monetario y cultural es-CR para la presentación en colones.

10. API REST

La API debe utilizar DTO, versionado, validaciones, documentación OpenAPI/Swagger, códigos HTTP correctos y respuestas ProblemDetails. No se deben exponer directamente las entidades de Entity Framework Core.

10.1. Endpoints mínimos

```
GET /api/v1/licitaciones
GET /api/v1/licitaciones/{id}
POST /api/v1/licitaciones
PUT /api/v1/licitaciones/{id}
PATCH /api/v1/licitaciones/{id}/estado
DELETE /api/v1/licitaciones/{id}
GET /api/v1/licitaciones/{id}/ofertas
POST /api/v1/licitaciones/{id}/ofertas
GET /api/v1/licitaciones/{id}/mejor-oferta

GET/POST/PUT/DELETE /api/v1/proveedores
GET/POST/PUT/DELETE /api/v1/ofertas
GET/POST/PUT/DELETE /api/v1/niveles-aprobacion
GET/POST/PUT/DELETE /api/v1/tipos-cambio
PATCH /api/v1/tipos-cambio/{id}/activar
```

10.2. Requisitos HTTP

- Listados con paginación, filtrado y ordenamiento.
- 200 OK, 201 Created, 204 No Content, 400 Bad Request, 404 Not Found, 409 Conflict, 422 Unprocessable Entity cuando corresponda y 500 mediante respuesta controlada.
- ProblemDetails con título, estado, detalle seguro, código de error e identificador de correlación.
- No exponer stack traces, rutas internas, consultas, secretos o mensajes técnicos al cliente.
- Colección reproducible de solicitudes de API documentada dentro de /docs.

11. Persistencia, auditoría y concurrencia

- Persistencia exclusiva con PostgreSQL local; SQLite no puede sustituirlo en la aplicación ni en las pruebas de integración.
- Entity Framework Core 9 y proveedor de PostgreSQL compatible.
- Migraciones versionadas y datos semilla para estados, niveles de aprobación y tipo de cambio inicial.
- Claves primarias, foráneas, índices únicos y restricciones CHECK.
- Cadena de conexión mediante variables de entorno o secretos; no se permiten credenciales reales en el repositorio.
- Campos CreatedAt y UpdatedAt; DeletedAt cuando se use borrado lógico.
- Control de concurrencia optimista mediante una columna de versión o mecanismo equivalente de PostgreSQL.
- Manejo controlado de DbUpdateConcurrencyException y DbUpdateException.
- Transacciones cuando una operación afecte varios registros relacionados.

12. Pruebas y desarrollo dirigido por pruebas (TDD)

TDD es una práctica obligatoria de XP. Las pruebas no se incorporarán únicamente al final del proyecto. El historial debe mostrar ciclos de prueba, implementación mínima y refactorización durante las iteraciones.

12.1. Pruebas Unitarias

- Presupuesto y oferta mayores que cero.
- Rechazo de oferta superior al presupuesto.
- Oferta duplicada.
- Estado no publicado.
- Vencimiento.
- Normalización y duplicidad de proveedor.
- Código único.
- Mejor oferta y desempate.
- Clasificación de ahorro.
- Nivel de aprobación.
- Conversión CRC/USD.
- Transiciones de estado.

12.2. Pruebas de Integración

- Ejecución contra PostgreSQL real en contenedor.
- Migraciones, índices únicos, claves foráneas, restricciones, transacciones y concurrencia.
- Persistencia y recuperación de datos.
- Pruebas de endpoints con infraestructura real cuando corresponda.

12.3. Pruebas Funcionales de Extremo a Extremo

- Landing page y navegación.
- Creación y edición de proveedor.
- Creación, publicación y cierre de licitación.
- Registro y rechazo de ofertas.
- Modo claro/oscuro.
- Conversión CRC/USD.
- Mensajes de validación.
- CRUD completo desde navegador.

12.4. Cobertura y calidad

La capa Domain y Application debe alcanzar al menos 80 % de cobertura de líneas; el proyecto completo debe alcanzar al menos 70 %. La cobertura numérica no sustituye la calidad de los escenarios probados.

13. Docker, Kubernetes e Integración Continua

13.1. Docker y Docker Compose

- Dockerfile multi-stage compatible con .NET 9.
- Usuario no privilegiado cuando sea viable.
- Servicio de aplicación y servicio PostgreSQL.
- Volumen persistente.
- Variables de entorno.
- Health checks.
- Inicio reproducible mediante docker compose up --build.
- Persistencia demostrable después de reiniciar contenedores.

13.2. Kubernetes

```
/k8s
namespace.yaml
app-deployment.yaml
app-service.yaml
app-configmap.yaml
app-secret.example.yaml
postgres-statefulset.yaml
postgres-service.yaml
postgres-pvc.yaml
```

- Deployment para la aplicación.
- StatefulSet o mecanismo adecuado para PostgreSQL.
- Service y PersistentVolumeClaim.

- ConfigMap y Secret.
- Startup, readiness y liveness probes.
- Solicitudes y límites de recursos.
- Migraciones ejecutadas de forma controlada.
- Evidencia de pods, servicios, PVC, logs y conservación de datos tras reinicio.

13.3. GitHub Actions

- Restaurar dependencias y compilar.
- Ejecutar pruebas y cobertura.
- Comprobar formato y análisis estático.
- Construir la imagen Docker.
- Validar manifiestos Kubernetes.
- Revisar dependencias vulnerables.
- Bloquear la integración de cambios cuando el flujo falle.

14. Git, GitHub y Entregas Incrementales

El historial debe demostrar el desarrollo incremental requerido por XP. No basta con cumplir una cantidad mínima de commits; se evaluará su distribución, contenido, relación con historias y coherencia técnica.

14.1. Entregas mínimas identificables

1. Inicialización y estructura.
2. Historias y planificación XP.
3. Dominio y modelo de datos.
4. Persistencia y migraciones.
5. Proveedores.
6. Licitaciones.
7. Ofertas.
8. Niveles de aprobación.
9. Conversión monetaria.
10. MVC y landing page.
11. API REST.
12. Pruebas.
13. Docker.
14. Kubernetes.
15. Documentación y cierre.

14.2. Convenciones

- Commits frecuentes, pequeños y con propósito técnico.
- Mensajes descriptivos; se recomienda Conventional Commits.
- No utilizar mensajes conversacionales, ambiguos o ajenos al cambio realizado.
- En parejas, ambos integrantes deben contribuir y alternar el trabajo; el historial no debe concentrarse en una sola cuenta.
- Cada historia debe vincularse con commits, pruebas o issues.
- La entrega evaluable debe quedar identificada mediante la etiqueta v1.0.0 o entrega-final.
- No subir secretos, archivos .env, binarios, carpetas generadas o credenciales.

feat (ofertas): impedir registro después del vencimiento
fix (proveedores): normalizar espacios duplicados
test (api): cubrir conflicto por código repetido
refactor (aprobacion): simplificar búsqueda de rangos
docs (xp): registrar resultados de la iteración 3

15. Documentación Interna en: */docs*

Única forma de documentación del proyecto:

No se entregarán documentos Word, PDF, PowerPoint, enlaces externos ni anexos separados. Toda la documentación creada por los estudiantes debe permanecer dentro del mismo repositorio, en la carpeta */docs* y en archivos Markdown (.md). Las imágenes o evidencias deberán almacenarse en */docs/assets* y enlazarse desde los archivos Markdown.

15.1. Estructura mínima requerida

```
/docs
README.md
vision-alcance.md
historias-usuario.md
plan-xp.md
bitacora-xp.md
arquitectura-general.md
modelo-datos.md
api.md
pruebas.md
docker.md
kubernetes.md
uso-ia.md
integracion-modulos.md
/modulos
licitaciones.md
proveedores.md
ofertas.md
niveles-aprobacion.md
tipo-cambio.md
interfaz-web.md
api-rest.md
persistencia.md
/assets
```

15.2. Contenido obligatorio

- */docs/README.md* funcionará como índice de navegación de toda la documentación.
- Cada módulo o servicio debe poseer un archivo Markdown independiente que explique propósito, responsabilidades, dependencias, entradas, salidas, reglas, errores y pruebas.
- *integracion-modulos.md* debe explicar cómo cooperan todos los módulos y servicios, describiendo los flujos de extremo a extremo y los límites entre componentes.
- *historias-usuario.md* debe contener historias, prioridad, estimación y criterios de aceptación.
- *plan-xp.md* debe documentar plan de liberación, iteraciones y reglas de trabajo XP.

- bitacora-xp.md debe registrar resultados, velocidad, retroalimentación, TDD, refactorizaciones y pequeñas liberaciones por iteración.
- arquitectura-general.md y modelo-datos.md deben incluir diagramas Mermaid o imágenes almacenadas en /docs/assets.
- pruebas.md debe explicar estrategia, ejecución, cobertura y casos principales.
- docker.md y kubernetes.md deben contener instrucciones reproducibles.
- api.md debe documentar endpoints, contratos, ejemplos y errores.

No se requiere documentación fuera de /docs. El archivo /docs/README.md sustituye el README documental tradicional ubicado en la raíz.

16. Uso responsable de herramientas de inteligencia artificial

- Se permite utilizar herramientas de IA como asistencia, siempre que su uso se declare en /docs/uso-ia.md.
- La declaración debe indicar herramienta, finalidad, módulos asistidos, ejemplos relevantes y validaciones realizadas por el estudiante.
- Los estudiantes son responsables de comprender, probar, corregir y defender todo el código entregado.
- No se aceptará “la IA lo generó” como explicación de una decisión o error.
- La persona docente puede solicitar explicación oral o modificación en vivo de cualquier parte del sistema.
- Una herramienta de IA no constituye un integrante adicional ni sustituye la programación en parejas.
- No deben insertarse comentarios artificiales, mensajes ocultos o contenido ajeno a la funcionalidad con el propósito de identificar una herramienta.

17. Entregables y condiciones de aceptación

17.1. Entregables

- Repositorio de GitHub accesible para evaluación.
- Código fuente completo.
- Migraciones y datos semilla.
- Pruebas unitarias, de integración y funcionales.
- Dockerfile y Docker Compose.
- Carpeta /k8s.
- GitHub Actions.
- Carpeta /docs con todos los archivos Markdown requeridos.
- Etiqueta de entrega final.

17.2. Condiciones de aceptación

Área	Condición
Compilación	La solución compila sin errores y sin advertencias evitables.
Pruebas	Las pruebas se ejecutan satisfactoriamente y cumplen la cobertura definida.
Base de datos	PostgreSQL inicia, aplica migraciones y conserva datos.
Docker	docker compose up --build inicia la solución sin pasos manuales complejos.

Área	Condición
Kubernetes	Aplicación y PostgreSQL se despliegan; probes y almacenamiento funcionan.
CI	El flujo de GitHub Actions asociado con la entrega final se encuentra satisfactorio.
Documentación	La carpeta /docs está completa, navegable y corresponde con la implementación.
XP	El repositorio evidencia iteraciones, Planning Game, TDD, integración, refactorización y pequeñas liberaciones.
Defensa	Cada integrante puede explicar y modificar la solución cuando sea requerido.

18. Rúbrica de Evaluación

La calificación total del proyecto es de 100 puntos. Todos los requisitos descritos forman parte de esta evaluación.

Criterio	Aspectos Evaluados	Puntos
1. Aplicación de Extreme Programming (XP)	Planning Game; historias y criterios; al menos cuatro iteraciones; pequeñas liberaciones; TDD; integración continua; diseño simple; refactorización; programación en parejas cuando corresponda; propiedad colectiva; ritmo sostenible; evidencias en /docs.	15
2. Lógica de negocio y validaciones	Estados, vencimiento, unicidad, normalización, montos positivos, límites presupuestarios, mejor oferta, clasificación, aprobación, moneda, integridad y mensajes controlados.	18
3. CRUD completo y API REST	CRUD de todas las entidades; DTO; versionado; OpenAPI; paginación; filtrado; códigos HTTP; ProblemDetails; seguridad de errores.	12
4. Arquitectura, modularidad y calidad del código	Separación de responsabilidades; diseño simple; inyección de dependencias; comentarios útiles; estándares; refactorización; dependencias justificadas; mantenibilidad.	10
5. PostgreSQL, modelo de datos, auditoría y concurrencia	EF Core; migraciones; relaciones; restricciones; índices; decimal; semilla; auditoría; transacciones; concurrencia; secretos.	10
6. Interfaz y experiencia de usuario	Landing page; navegación; formularios; calendario; modo claro/oscuro; CRC/USD; accesibilidad; diseño adaptable; recursos frontend reproducibles.	8
7. TDD y pruebas automatizadas	Pruebas unitarias, integración con PostgreSQL real, funcionales E2E, escenarios relevantes, cobertura y ejecución reproducible.	12
8. Docker, Kubernetes e integración continua	Dockerfile; Compose; persistencia; health checks; manifiestos completos; probes; PVC; configuración; evidencias de despliegue; GitHub Actions.	10
9. Git, GitHub y documentación interna	Commits incrementales; participación; issues/historias; etiqueta final; higiene del repositorio; /docs completa; documentación por módulo e integración general.	5

TOTAL 100

Criterio de Trazabilidad

Cada elemento evaluado debe poder relacionarse con una historia de usuario, una prueba, uno o más commits y la documentación correspondiente dentro de /docs. La ausencia de trazabilidad limita la posibilidad de comprobar el proceso XP y el cumplimiento técnico.